

Housing Price Prediction Application

Application Overview

This application is a containerized microservice written in Go that predicts residential sale prices based on characteristics like square footage, lot size, and build year, using a linear regression model trained on historical housing sales data from the Kaggle Housing Prices dataset. The application runs on github actions and is deployed using an AWS EC2 instance. Through this instance, the application can import a json with the features of a hypothetical house and predict its sale price. A monitoring dashboard tracks the application's performance in real time, showing information such as the number of requests being handled, the accuracy of its predictions, latency, and the error rate from malformed requests.

Advantages

- Fast and lightweight: the service starts quickly and runs efficiently, with no software dependencies to install or manage.
- Easy to extend: the variables the model uses when predicting a price are kept in a central list, so adding or removing variables to that list is quick and easy.
- Built-in monitoring: a live dashboard using Grafana shows how the service is performing, and how good its predictions are
- Safe updates: github actions tests any updates before deploying, so issues can be caught before being pushed to deployment.
- Low cost: the entire application runs on a small, inexpensive cloud instance

Disadvantages

- Simple prediction method: the current model uses a simple linear regression model, so it can miss more complex patterns that a more advanced ML model would catch. For example, some features may interact with each to affect the price in a way that a linear regression does not account for
- Ignores non-numeric details: the model only considers numerical variables like square footage, and not descriptive details like neighborhood or appearance, which could have effects on the sale price
- No backup: the application runs on a single instance, so an outage on that instance will take down the application
- Doesn't update: the model is trained once and does not update itself with new sales data. All updates must be done manually.

Areas for Improvement

- Implement a different ML model that fits the Kaggle dataset better.
- Include descriptive details to the prediction model, which should meaningfully improve prediction accuracy
- Implement a testing dataset to see how well the model predicts data it wasn't trained on
- Set up automatic retraining so the model can be updated quickly as new sales data is acquired, instead of needing to manually re-train the model

New Technologies to Consider

- More advanced prediction models (such as XGBoost): These models can produce more accurate predictions than the current method, especially for data with many interacting features
- Managed hosting platforms (such as Amazon ECS): ECS would take over the work of running and scaling the servers running the application, reducing the amount of manual setup and maintenance required
- Managed monitoring tools (such as Amazon Managed Grafana): This would provide the same kinds of dashboards currently in use without needing to manage Grafana separately, as it is already integrated into the AWS service
- Infrastructure automation tools (such as Terraform): this would allow the entire setup to be recreated automatically and consistently, rather than configured by hand each time.

Conclusion

This application runs well as a first version and proof of concept, taking raw housing sales data, using it to train a ML model, using that model to make predictions, then deploying that model to a monitored microservice using cloud-based platforms. The main issues with the application (simple prediction model, no backup, and manual updating) can be improved by adding to the existing application, rather than starting over from the beginning. The first step towards improving this service would be to focus on improving prediction accuracy by changing the model used and implementing descriptive data in addition to the numerical data.

Github Repository

<https://github.com/sdnkearns/Housing-Microservice-Project-434>